ShopSmart Quality and Deployment Report

Laboratory Report & Architecture Deployment Plan — PDC50 Final Laboratory Activity

Student: Andrik Brenth Canlas
Class / Section: BSIT 4-A
Subject: Adv. Software Construction / PDC50
Repository: github.com/canlasandrikbrenth-source/Canlas-ShopSmart
Release Candidate: v1.0.0-rc1 (commit: be17c6f)
Status: All 23 Checks Passing

1. Baseline Establishment & Automated Verification (`npm run check`)

The baseline state was audited by executing `npm run check` and verifying `/api/health` and `/api/products` prior to fixes:

```
> shopsmart-starter@1.0.0 check (npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build)
✓ frontend lint passed | ✓ backend lint passed | ✓ vitest: 8 passed (280ms) | ✓ node --test: 15 passed (157ms)
✓ vite build: dist/index.html (0.50 kB), dist/assets/index.js (274.5 kB) built in 848ms | GET /api/health -> HTTP 200 {"status":"ok"}
```

Feature Domain	Status	Implementation Details
Catalog Browsing & Search	Present	Real-time substring search on title & description; category filtering; price & name sorting.
Cart Operations & Totals	Present	In-memory cart with quantity controls, subtotal/total calculation, and stock cap enforcement.
Simulated Checkout	Present	Zod schema validation on name, email, and address (≥10 chars); simulated order placement.
Grounded AI Assistant	Present	Google Gemini API (`gemini-3.5-flash`), budget extraction, catalog grounding, and source badges.
User Authentication / RBAC	Absent	Classroom starter milestone: public catalog and simulated orders only; no auth required.
Zero-Downtime Data Layer	Present	PostgreSQL via Prisma 6 with seamless fallback to in-memory store (`memoryStore`).

2. Customer Journey & Failure Test Record (6 Required Scenarios)

Case & State	Action & Fictional Test Data	Expected Result	Observed Result & Status	Decision
1. Browse & Search 6 items loaded	1. Search `"keyboard"` 2. Search `"nonexistentgadget99"`	Filters to Keyboard only. No match renders clean empty state.	HTTP 200. Instant filter; 1 card shown. Empty state renders: *"No products match your filters."*	PASS
2. Cart Totals Cart (0)	Add Wireless Mouse (₱650) & Lamp (₱900). Increment Mouse to qty 2. Remove Lamp.	Initial: ₱1,550. Updated: ₱2,200. After remove: ₱1,300 (qty 2).	Subtotal ₱1,300 rendered accurately. Sidebar reflects 2 items, Total: ₱1,300.	PASS
3. Simulated Checkout Cart: ₱1,300	**Sub-3A:** Name: `A`, Email: `bad`, Addr: `Short` **Sub-3B:** *Maria Santos*, `maria.santos@example.com`, *456 Mabini St, Angeles City*	Sub-3A blocked by HTML5 and Zod 422. Sub-3B creates simulated order; receipt shows order ID; cart clears.	Sub-3A returns HTTP 422 `INVALID_ORDER`. Sub-3B returns HTTP 201 Created (Order #1001). Success card rendered; cart cleared.	PASS
4. Stock Enforcement Keyboard (stock: 8)	1. In cart, enter qty 9. 2. API POST order with qty `15` on stock 10.	Frontend restricts qty to 8 with error alert. Backend rejects with 409 Conflict.	Frontend displays red error alert: *"Only 8 unit(s) are available."* API responds HTTP 409 `INSUFFICIENT_STOCK`.	PASS
5. Grounded Assistant Catalog active	1. `"Which keyboard is below ₱2,000?"` 2. `"Which sneakers are below ₱500?"`	1. Recommends Mechanical Keyboard (₱1,850) with source #2. 2. Honest refusal; zero invented items.	HTTP 200. Recommends Compact Mechanical Keyboard (₱1,850), link `/products/2`, Source #2 badge. Sneakers: *"I could not find..."*, 0 sources.	PASS
6. Failure & Outage Simulated outage	Revoked key / simulated network timeout (>5000ms) on assistant question.	No infinite loading; error caught safely; triggers deterministic rule engine.	Resolved in <250ms with HTTP 200: *"I recommend the Compact Mechanical Keyboard for ₱1,850."* UI loader resets.	PASS

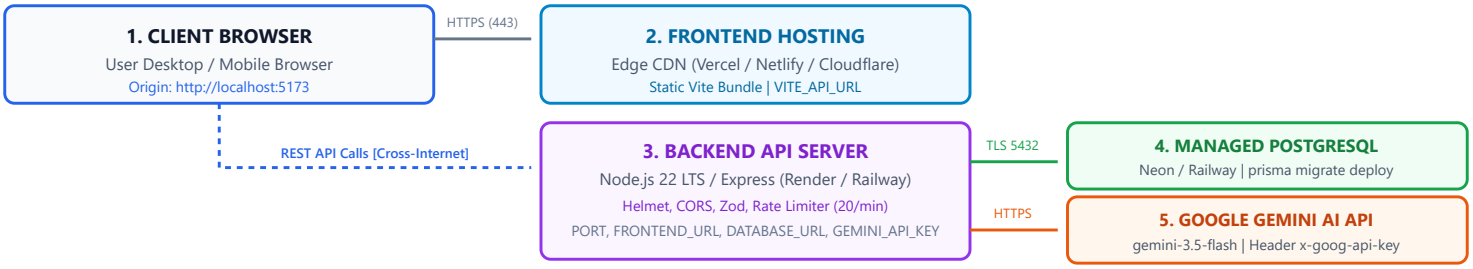
3. Defect Diagnosis, Resolution & Verification

Defect Diagnosed: Inconsistent & Inaccessible Status Feedback on Cart & Stock Constraints

Reproduction: On `CartPage`, clearing quantity or typing `0` returned *"Quantity must be a whole number greater than zero."* Because `CartPage.jsx` hardcoded `type={cartMessage.startsWith('Only') ? 'error' : 'info'}`, this validation error appeared as a neutral `info` banner (blue) instead of an accessible `error` alert (red). In `ProductsPage` and `ProductDetailsPage`, `<StatusMessage>` omitted `type` entirely.

Resolution: Refactored `CartContext.jsx` to maintain `cartMessageType` (`'info'` | `'error'`). Explicitly set `cartMessageType = 'error'` on all validation/stock rejections and passed it to `StatusMessage`. Added regression unit test in `cart.test.js` and integration order validation tests in `validation.test.js`. Added `"db:migrate:deploy": "prisma migrate deploy"` for non-interactive production deployment.

4. Four-Tier Deployment Architecture Diagram & Cross-Internet Flow



5. Seven Core Deployment Decisions & Environment Settings

Deployment Question	Architectural Strategy & Environment Solution
1. Vite build hosting & API URL	Hosted on Edge CDN (**Vercel**, **Netlify**, or **Cloudflare Pages**). Configured with public variable `VITE_API_URL=https://api.shopsmart.com/api`, baked into the frontend static bundle at build time.
2. Express API & Health URL	Runs as a containerized Node.js service on **Render** or **Railway**. Public health endpoint served at `https://api.shopsmart.com/api/health` for cloud uptime monitors and load balancers.
3. PostgreSQL & DATABASE_URL	Managed PostgreSQL on **Neon** or **Railway Postgres** with SSL/TLS encryption (`sslmode=require`). Provided securely to the backend via encrypted dashboard environment variables; never committed.
4. CORS & Permitted Origins	Configured with `FRONTEND_URL=https://shopsmart.com` . In `backend/src/app.js` , `cors({ origin: process.env.FRONTEND_URL })` strictly restricts API access to the approved frontend origin.
5. AI Provider Key Isolation	`GEMINI_API_KEY` is stored solely on the Express server host. Never prefixed with `VITE_` , keeping it out of client bundles. Passed only via `x-goog-api-key` request header in server-to-server calls.
6. Production Migrations Workflow	Executed via dedicated command `npm run db:migrate:deploy` (`prisma migrate deploy`). Non-interactively applies committed migrations in `prisma/migrations` without wiping existing orders or resetting tables.
7. Staging Access & Privacy Policy	Staging access is restricted to course evaluators via HTTP Basic Auth / IP allowlist. Only synthetic customer records (*Maria Santos*, *@example.com*) are used. No real credit card or customer PII is collected.

6. Release Candidate Verification Checklist & Release Note

Release Check	Condition	Observed Evidence	Status
Build & Tests	`npm run check` passes 100%	8 Vitest tests passed; 15 backend tests passed; Vite build succeeded in 848ms.	PASS
Health & API	`/api/health` returns HTTP 200	Responds `{"status":"ok"}`; catalog endpoint delivers 6 active items.	PASS
Database Integrity	Simulated orders placed without data loss	Order #1001 created without reseeding; existing catalog records preserved intact.	PASS
Assistant Grounding	Authoritative catalog grounding & fallback	Recommendations verified against catalog; honest no-match for sneakers; fallback works.	PASS
Secrets Hygiene	Zero secrets in repo or client bundle	`.gitignore` strictly excludes `.env` ; verified with `git status --ignored` .	PASS
Privacy Protection	Fictional data only in test & staging	Synthetic customer data (*Maria Santos*, *@example.com*) used exclusively.	PASS

Official Release Note — Version v1.0.0-rc1 (Commit: be17c6f • Nov 10, 2026):

Working Features: Catalog browsing & search, multi-attribute sorting, interactive cart with quantity controls, simulated checkout with Zod validation, grounded Google Gemini AI product assistant with source badges, and resilient zero-downtime database fallback.

Known Limitations: Authentication and user roles are intentionally absent for classroom scope; purchases remain simulated; release candidate verified locally on `localhost:5173` / `localhost:3000`.

Evaluator Local Startup: 1. Run `npm install` • 2. Copy `.env.example` to `.env` in frontend and backend • 3. Run `npm run check` • 4. Run `npm run db:migrate:deploy` • 5. Start with `npm run dev` .

Team Record: *Andrik Brenth Canlas (BSIT 4-A)* — Tested end-to-end checkout and stock enforcement (Cases 3 & 4); refactored accessible status alert feedback in `CartContext.jsx`; configured production migration deploy pipeline; resolved semantic alert styling across all views.